Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment 2: DDL and DML queries

Name : Om Rajendra Bharambe
PRN : 124B1F001
Batch : A1
Date : 10/02/2026

Aim:

Use Create, Select, Update, Delete with operators, functions, and set operator:

a. Design and Develop SQL DDL statements which demonstrate the use of SQL objects such as Table, Index and different constraints etc.
b. Write at least 10 SQL queries on the suitable database application using SQL DML Statements.

Objectives:

- **To design and develop** SQL DDL statements that demonstrate the use of structural objects such as tables and indexes while enforcing data integrity through various constraints.
- **To implement and execute** a comprehensive set of DML and DQL queries, including INSERT, UPDATE, DELETE, and SELECT, to effectively manage and retrieve data within a database application.
- **To manage schema evolution** by utilizing the ALTER command to modify existing table structures and ensure the database adapts to changing system requirements without data loss.

Case Study: Airline Management System

SQL Queries:

Initial Setup: Server Login & Database Creation

```
Enter password: *******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 18
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
| world              |
+--------------------+
6 rows in set (0.02 sec)

mysql> CREATE DATABASE airlines;
Query OK, 1 row affected (0.01 sec)

mysql> USE airlines;
Database changed
```

DDL Query: Creating the Airline Table

Data Definition Language (DDL) is used to define or modify the database structure. Here, we are creating our first table and verifying its structure.

```
mysql> CREATE TABLE Airline(
    -> airline_id INT PRIMARY KEY,
    -> airline_name VARCHAR(50) NOT NULL,
    -> country VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> DESC Airline;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| airline_id   | int         | NO   | PRI | NULL    |       |
| airline_name | varchar(50) | NO   |     | NULL    |       |
| country      | varchar(50) | YES  |     | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
3 rows in set (0.02 sec)
```

DML Query: Inserting Data into Airline

Data Manipulation Language (DML) is used to add, modify, or delete data. Followed by a Data Query Language (DQL) command to view the inserted records.

```
mysql> INSERT INTO Airline VALUES
    -> (1,'Air India','India'),
    -> (2,'Indigo','India'),
    -> (3,'SpiceJet','India'),
    -> (4,'Emirates','UAE');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
mysql> SELECT * FROM Airline;
+------------+--------------+---------+
| airline_id | airline_name | country |
+------------+--------------+---------+
|          1 | Air India    | India   |
|          2 | Indigo       | India   |
|          3 | SpiceJet     | India   |
|          4 | Emirates     | UAE     |
+------------+--------------+---------+
4 rows in set (0.00 sec)
```

DDL Query: Creating the Flight Table

Creating a child table with a Foreign Key constraint linking back to the Airline table.

```
mysql> CREATE TABLE Flight(
    -> flight_id INT PRIMARY KEY,
    -> flight_name VARCHAR(50),
    -> source VARCHAR(50),
    -> destination VARCHAR(50),
    -> price INT CHECK(price>0),
    -> airline_id INT,
    -> FOREIGN KEY (airline_id) REFERENCES Airline(airline_id)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> DESC Flight;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| flight_id   | int         | NO   | PRI | NULL    |       |
| flight_name | varchar(50) | YES  |     | NULL    |       |
| source      | varchar(50) | YES  |     | NULL    |       |
| destination | varchar(50) | YES  |     | NULL    |       |
| price       | int         | YES  |     | NULL    |       |
| airline_id  | int         | YES  | MUL | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
6 rows in set (0.01 sec)
```

DML Query: Inserting Data into Flight

```
mysql> INSERT INTO Flight VALUES
    -> (101,'AI101','Mumbai','Delhi',5000,1),
    -> (102,'IN202','Pune','Bangalore',4500,2),
    -> (103,'SJ303','Delhi','Goa',6000,3),
    -> (104,'EM404','Dubai','Mumbai',15000,4);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Flight;
+-----------+-------------+--------+-------------+-------+------------+
| flight_id | flight_name | source | destination | price | airline_id |
+-----------+-------------+--------+-------------+-------+------------+
|       101 | AI101       | Mumbai | Delhi       |  5000 |          1 |
|       102 | IN202       | Pune   | Bangalore   |  4500 |          2 |
|       103 | SJ303       | Delhi  | Goa         |  6000 |          3 |
|       104 | EM404       | Dubai  | Mumbai      | 15000 |          4 |
+-----------+-------------+--------+-------------+-------+------------+
```

```
4 rows in set (0.00 sec)
```

DDL & DML Queries: Setting up the Passenger Table

Defining the schema and populating it with passenger data.

```
mysql> CREATE TABLE Passenger(
    -> passenger_id INT PRIMARY KEY,
    -> passenger_name VARCHAR(50),
    -> gender CHAR(1) CHECK(gender IN('M','F')),
    -> age INT CHECK(age>0)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> DESC Passenger;
+----------------+-------------+------+-----+---------+-------+
| Field          | Type        | Null | Key | Default | Extra |
+----------------+-------------+------+-----+---------+-------+
| passenger_id   | int         | NO   | PRI | NULL    |       |
| passenger_name | varchar(50) | YES  |     | NULL    |       |
| gender         | char(1)     | YES  |     | NULL    |       |
| age            | int         | YES  |     | NULL    |       |
+----------------+-------------+------+-----+---------+-------+
4 rows in set (0.01 sec)

mysql> INSERT INTO Passenger VALUES
    -> (1,'Rahul Sharma','M',28),
    -> (2,'Neha Verma','F',25),
    -> (3,'Amit Patil','M',32),
    -> (4,'Sneha Kulkarni','F',27);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Passenger;
+--------------+----------------+--------+------+
| passenger_id | passenger_name | gender | age  |
+--------------+----------------+--------+------+
|            1 | Rahul Sharma   | M      |   28 |
|            2 | Neha Verma     | F      |   25 |
|            3 | Amit Patil     | M      |   32 |
|            4 | Sneha Kulkarni | F      |   27 |
+--------------+----------------+--------+------+
4 rows in set (0.00 sec)
```

DDL Query: Creating the Booking Table

```
mysql> CREATE TABLE Booking(
    -> booking_id INT PRIMARY KEY,
    -> passenger_id INT,
    -> flight_id INT,
    -> booking_date DATE DEFAULT (CURRENT_DATE),
    -> seat_no VARCHAR(5),
    -> FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id),
    -> FOREIGN KEY (flight_id) REFERENCES Flight(flight_id)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> DESC Booking;
+--------------+------------+------+-----+-----------+-------------------+
| Field        | Type       | Null | Key | Default   | Extra             |
+--------------+------------+------+-----+-----------+-------------------+
| booking_id   | int        | NO   | PRI | NULL      |                   |
| passenger_id | int        | YES  | MUL | NULL      |                   |
| flight_id    | int        | YES  | MUL | NULL      |                   |
| booking_date | date       | YES  |     | curdate() | DEFAULT_GENERATED |
| seat_no      | varchar(5) | YES  |     | NULL      |                   |
+--------------+------------+------+-----+-----------+-------------------+
5 rows in set (0.01 sec)
```

DML Query: Inserting Data into Booking

```
mysql> INSERT INTO Booking(booking_id,passenger_id,flight_id,seat_no) VALUES
    -> (1,1,101,'12A'),
    -> (2,2,102,'14B'),
    -> (3,3,103,'10C'),
    -> (4,4,104,'8D');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Booking;
+------------+--------------+-----------+--------------+---------+
| booking_id | passenger_id | flight_id | booking_date | seat_no |
+------------+--------------+-----------+--------------+---------+
|          1 |            1 |       101 | 2026-03-04   | 12A     |
|          2 |            2 |       102 | 2026-03-04   | 14B     |
|          3 |            3 |       103 | 2026-03-04   | 10C     |
|          4 |            4 |       104 | 2026-03-04   | 8D      |
+------------+--------------+-----------+--------------+---------+
4 rows in set (0.00 sec)
```

DQL Query: Selecting Specific Columns and Filtering (WHERE clause)

Data Query Language (DQL) commands to extract precise information rather than fetching whole tables.

```
mysql> SELECT airline_name,country FROM Airline;
+--------------+---------+
| airline_name | country |
+--------------+---------+
| Air India    | India   |
| Indigo       | India   |
| SpiceJet     | India   |
| Emirates     | UAE     |
+--------------+---------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Flight
    -> WHERE price > 5000;
+-----------+-------------+--------+-------------+-------+------------+
| flight_id | flight_name | source | destination | price | airline_id |
+-----------+-------------+--------+-------------+-------+------------+
|       103 | SJ303       | Delhi  | Goa         |  6000 |          3 |
|       104 | EM404       | Dubai  | Mumbai      | 15000 |          4 |
+-----------+-------------+--------+-------------+-------+------------+
2 rows in set (0.01 sec)
```

DDL Queries: Using ALTER TABLE to Modify Schema

These commands demonstrate changing the database structure after it has been created: adding a column, modifying a data type, and renaming a column.

```
mysql> ALTER TABLE Passenger
    -> ADD phone VARCHAR(15);
Query OK, 0 rows affected (0.05 sec)

mysql> DESC Passenger;
+----------------+-------------+------+-----+---------+-------+
| Field          | Type        | Null | Key | Default | Extra |
+----------------+-------------+------+-----+---------+-------+
| passenger_id   | int         | NO   | PRI | NULL    |       |
| passenger_name | varchar(50) | YES  |     | NULL    |       |
| gender         | char(1)     | YES  |     | NULL    |       |
| age            | int         | YES  |     | NULL    |       |
| phone          | varchar(15) | YES  |     | NULL    |       |
+----------------+-------------+------+-----+---------+-------+
```

```
5 rows in set (0.00 sec)

mysql> ALTER TABLE Flight
    -> MODIFY price DECIMAL(10,2);
Query OK, 4 rows affected (0.09 sec)

mysql> DESC Flight;
+-------------+---------------+------+-----+---------+-------+
| Field       | Type          | Null | Key | Default | Extra |
+-------------+---------------+------+-----+---------+-------+
| flight_id   | int           | NO   | PRI | NULL    |       |
| flight_name | varchar(50)   | YES  |     | NULL    |       |
| source      | varchar(50)   | YES  |     | NULL    |       |
| destination | varchar(50)   | YES  |     | NULL    |       |
| price       | decimal(10,2) | YES  |     | NULL    |       |
| airline_id  | int           | YES  | MUL | NULL    |       |
+-------------+---------------+------+-----+---------+-------+
6 rows in set (0.00 sec)

mysql> ALTER TABLE Passenger
    -> RENAME COLUMN passenger_name TO pname;
Query OK, 0 rows affected (0.03 sec)

mysql> DESC Passenger;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| passenger_id | int         | NO   | PRI | NULL    |       |
| pname        | varchar(50) | YES  |     | NULL    |       |
| gender       | char(1)     | YES  |     | NULL    |       |
| age          | int         | YES  |     | NULL    |       |
| phone        | varchar(15) | YES  |     | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

DML Query: Updating Existing Records

Using **UPDATE** to change the specific value of a record currently stored in the database.

```
mysql> UPDATE Flight
    -> SET price = 5500
    -> WHERE flight_id = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Flight;
+-----------+-------------+--------+-------------+----------+------------+
| flight_id | flight_name | source | destination | price    | airline_id |
+-----------+-------------+--------+-------------+----------+------------+
|       101 | AI101       | Mumbai | Delhi       |  5500.00 |          1 |
|       102 | IN202       | Pune   | Bangalore   |  4500.00 |          2 |
|       103 | SJ303       | Delhi  | Goa         |  6000.00 |          3 |
|       104 | EM404       | Dubai  | Mumbai      | 15000.00 |          4 |
+-----------+-------------+--------+-------------+----------+------------+
4 rows in set (0.00 sec)
```

DML Query: Deleting Records

Using **DELETE** to remove a specific record from a table entirely.

```
mysql> DELETE FROM Booking
    -> WHERE booking_id = 3;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Booking;
+------------+--------------+-----------+--------------+---------+
| booking_id | passenger_id | flight_id | booking_date | seat_no |
+------------+--------------+-----------+--------------+---------+
|          1 |            1 |       101 | 2026-03-04   | 12A     |
|          2 |            2 |       102 | 2026-03-04   | 14B     |
|          4 |            4 |       104 | 2026-03-04   | 8D      |
+------------+--------------+-----------+--------------+---------+
3 rows in set (0.00 sec)

mysql>
```

Case Study: Civic Voice Interface System

SQL Queries:

Initial Setup: Server Login & Database Creation

```
Enter password: *******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| airlinecomp        |
| airlines           |
| banksystem         |
| fkpk               |
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
| world              |
+--------------------+
10 rows in set (0.08 sec)

mysql> CREATE DATABASE civic_voice;
Query OK, 1 row affected (0.02 sec)

mysql> USE civic_voice;
Database changed
```

DDL Queries: Building the Schema

Data Definition Language (DDL) commands. Here we are defining the entire relational structure of your database by creating four linked tables: **User**, **CitizenProfile**, **UserDocument**, and **Complaint**.

```
mysql> CREATE TABLE User(
    ->     user_id INT PRIMARY KEY,
    ->     email VARCHAR(100) NOT NULL UNIQUE,
    ->     name VARCHAR(50),
    ->     mobile VARCHAR(15),
    ->     language VARCHAR(20) DEFAULT 'English',
    ->     is_guest BOOLEAN DEFAULT FALSE
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE CitizenProfile(
    ->     profile_id INT PRIMARY KEY,
    ->     user_id INT,
    ->     full_name VARCHAR(100) NOT NULL,
    ->     state VARCHAR(50),
    ->     district VARCHAR(50),
    ->     pincode VARCHAR(10),
    ->     gender CHAR(1) CHECK(gender IN('M','F','O')),
    ->     age INT CHECK(age>0),
    ->     FOREIGN KEY (user_id) REFERENCES User(user_id)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE UserDocument(
    ->     doc_id INT PRIMARY KEY,
    ->     user_id INT,
    ->     doc_name VARCHAR(100) NOT NULL,
    ->     category VARCHAR(50),
    ->     status VARCHAR(20),
    ->     is_verified BOOLEAN DEFAULT FALSE,
    ->     FOREIGN KEY (user_id) REFERENCES User(user_id)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Complaint(
    ->     complaint_id INT PRIMARY KEY,
    ->     user_id INT,
    ->     title VARCHAR(100),
    ->     description TEXT,
    ->     status VARCHAR(20) DEFAULT 'Pending',
    ->     FOREIGN KEY (user_id) REFERENCES User(user_id)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

DML Queries: Populating the Tables

Data Manipulation Language (DML) commands to populate our newly created tables with initial sample records.

```
mysql> INSERT INTO User VALUES
    -> (1, 'rahul.sharma@email.com', 'Rahul Sharma', '9876543210', 'English', FALSE),
    -> (2, 'neha.verma@email.com', 'Neha Verma', '8765432109', 'Hindi', FALSE),
    -> (3, 'amit.patil@email.com', 'Amit Patil', '7654321098', 'Marathi', FALSE);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO CitizenProfile VALUES
    -> (101, 1, 'Rahul S. Sharma', 'Maharashtra', 'Pune', '411001', 'M', 28),
    -> (102, 2, 'Neha Verma', 'Delhi', 'New Delhi', '110001', 'F', 25),
    -> (103, 3, 'Amit Patil', 'Maharashtra', 'Mumbai', '400001', 'M', 32);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO UserDocument VALUES
    -> (1001, 1, 'Aadhaar Card', 'ID Proof', 'Approved', TRUE),
    -> (1002, 1, 'PAN Card', 'Identity', 'Pending', FALSE),
    -> (1003, 2, 'Driving License', 'ID Proof', 'Approved', TRUE),
    -> (1004, 3, 'Ration Card', 'Address Proof', 'Rejected', FALSE);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

DQL Queries: Retrieving and Filtering Data

Data Query Language (DQL) commands used to verify the insertions. This includes fetching an entire table, selecting specific columns, and using a **WHERE** clause to filter for unverified documents.

```
mysql> SELECT * FROM User;
+---------+-----------------------+--------------+------------+----------+----------+
| user_id | email                 | name         | mobile     | language | is_guest |
+---------+-----------------------+--------------+------------+----------+----------+
|       1 | rahul.sharma@email.com | Rahul Sharma | 9876543210 | English  |        0 |
|       2 | neha.verma@email.com   | Neha Verma   | 8765432109 | Hindi    |        0 |
|       3 | amit.patil@email.com   | Amit Patil   | 7654321098 | Marathi  |        0 |
+---------+-----------------------+--------------+------------+----------+----------+
3 rows in set (0.00 sec)

mysql> SELECT full_name, state, district FROM CitizenProfile;
+----------------+-------------+-----------+
| full_name      | state       | district  |
```

```
+-----------------+-------------+-----------+
| Rahul S. Sharma | Maharashtra | Pune      |
| Neha Verma      | Delhi       | New Delhi |
| Amit Patil      | Maharashtra | Mumbai    |
+-----------------+-------------+-----------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM UserDocument WHERE is_verified = FALSE;
+--------+---------+-------------+---------------+----------+-------------+
| doc_id | user_id | doc_name    | category      | status   | is_verified |
+--------+---------+-------------+---------------+----------+-------------+
|   1002 |       1 | PAN Card    | Identity      | Pending  |           0 |
|   1004 |       3 | Ration Card | Address Proof | Rejected |           0 |
+--------+---------+-------------+---------------+----------+-------------+
2 rows in set (0.00 sec)
```

DDL Queries: Modifying the Schema with ALTER

Using **ALTER TABLE** commands to adjust the table structures after the fact: adding a new column with a default value, increasing the size of a **VARCHAR** column, and renaming an existing column.

```
mysql> ALTER TABLE Complaint
    -> ADD created_date DATE DEFAULT (CURRENT_DATE);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE UserDocument
    -> MODIFY status VARCHAR(50);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE User
    -> RENAME COLUMN mobile TO phone_number;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

DML Queries: Updating and Deleting Records

Final DML operations to change the status of an existing document using **UPDATE** and remove a rejected document entirely using **DELETE**, followed by **SELECT** queries to verify the changes.

```
mysql> UPDATE UserDocument
    -> SET status = 'Approved', is_verified = TRUE
    -> WHERE doc_id = 1002;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM UserDocument WHERE doc_id = 1002;
+--------+---------+----------+----------+----------+-------------+
| doc_id | user_id | doc_name | category | status   | is_verified |
+--------+---------+----------+----------+----------+-------------+
|   1002 |       1 | PAN Card | Identity | Approved |           1 |
+--------+---------+----------+----------+----------+-------------+
1 row in set (0.00 sec)

mysql> DELETE FROM UserDocument
    -> WHERE doc_id = 1004;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM UserDocument;
+--------+---------+----------------+----------+----------+-------------+
| doc_id | user_id | doc_name       | category | status   | is_verified |
+--------+---------+----------------+----------+----------+-------------+
|   1001 |       1 | Aadhaar Card   | ID Proof | Approved |           1 |
|   1002 |       1 | PAN Card       | Identity | Approved |           1 |
|   1003 |       2 | Driving License | ID Proof | Approved |           1 |
+--------+---------+----------------+----------+----------+-------------+
3 rows in set (0.00 sec)

mysql>
```

Conclusion:

The successful implementation of this assignment demonstrates a comprehensive understanding of relational database management using SQL. By designing and executing DDL statements for both the **Airline Management System** and the **Civic Voice Interface**, the foundational structures of tables were established with critical integrity constraints such as **PRIMARY KEY**, **FOREIGN KEY**, and **CHECK** to ensure data reliability. Furthermore, the application of DML queries-including **INSERT**, **UPDATE**, and **DELETE**-facilitated effective data manipulation, while the use of **ALTER** commands proved essential for schema evolution. Ultimately, these exercises highlight how a well-structured relational model, maintained through precise SQL commands, ensures logical connectivity between entities and maintains the overall consistency required for enterprise-grade applications.